

UNIT 1

Arrays in C

1. What is an array in C? How is it different from a normal variable?

An array in C is a collection of a **finite number of homogeneous elements** stored in **contiguous memory locations** and accessed using a **single variable name with an index**. Arrays are used to store and process multiple values of the same data type efficiently.

A **normal variable** stores **only one value** at a time and occupies a **single memory location**.

Differences:

- An array can store **multiple values**, whereas a normal variable stores **only one value**.
- Array elements are accessed using **index (subscript)**, while a normal variable is accessed directly by its name.
- Arrays occupy **contiguous memory locations**, whereas a normal variable occupies **one memory location**.
- Arrays require loops for processing elements, while normal variables do not.

Example:

```
int a[5] = {10, 20, 30, 40, 50}; // array
int x = 10;                       // normal variable
```

Conclusion:

An array is used when multiple values of the same type need to be stored and processed efficiently, while a normal variable is suitable for storing a single value.

2. What is the difference between a one-dimensional and two-dimensional array?

A **one-dimensional array** stores elements in a **single linear sequence** and is accessed using **one index**, whereas a **two-dimensional array** stores elements in the form of **rows and columns** and is accessed using **two indices**.

Differences:

- A one-dimensional array represents a **list of elements**, while a two-dimensional array represents a **table or matrix**.
- One-dimensional arrays require **one subscript** for accessing elements, whereas two-dimensional arrays require **two subscripts**.
- Memory representation of a one-dimensional array is **linear**, whereas a two-dimensional array is stored in **row-major order** in memory.
- One-dimensional arrays are simpler and used for **linear data**, while two-dimensional arrays are used for **matrix-based data**.

Example:

```
int a[5];    // one-dimensional array
int b[3][3]; // two-dimensional array
```

Conclusion:

One-dimensional arrays are suitable for storing linear data, whereas two-dimensional arrays are used when data needs to be represented in rows and columns.

3. What is the difference between a character array and a string in C?

A **character array** is an array of characters that may or may not represent text, whereas a **string** in C is a **character array terminated by a null character ('**\0**')**.

Differences:

- A character array can store **any collection of characters**, but a string specifically stores **text data**.
- A string always ends with a **null character ('**\0**')**, while a character array **may not** include it.
- String-handling functions in C (like **strlen()**, **strcpy()**) work **only with strings**, not with plain character arrays.

- The size of a string is determined up to the **null character**, whereas a character array size is fixed by its declaration.

Example:

```
char a[] = {'H','e','l','l','o'};    // character array
char b[] = "Hello";                  // string (ends with '\0')
```

Conclusion:

All strings are character arrays, but not all character arrays are strings.

4. Write a C program to find the largest and smallest elements in an array.

```
#include <stdio.h>

int main() {
    int a[50], n, i;
    int largest, smallest;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    largest = smallest = a[0];

    for(i = 1; i < n; i++) {
        if(a[i] > largest)
            largest = a[i];
        if(a[i] < smallest)
            smallest = a[i];
    }

    printf("Largest element = %d\n", largest);
    printf("Smallest element = %d\n", smallest);
}
```

```
    return 0;
}
```

5. Write a program to reverse the elements of an array.

```
#include <stdio.h>

int main() {
    int a[50], n, i, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for(i = 0; i < n / 2; i++) {
        temp = a[i];
        a[i] = a[n - i - 1];
        a[n - i - 1] = temp;
    }

    printf("Reversed array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

6. Differentiate between row-major and column-major order in 2D arrays.

In a **two-dimensional array**, elements are stored in memory in a specific order. The two common storage orders are **row-major** and **column-major**.

Row-major order stores all the elements of a row **contiguously** in memory, row by row.

Column-major order stores all the elements of a column **contiguously**, column by column.

Differences:

- In **row-major order**, memory is filled **row-wise**, whereas in **column-major order**, memory is filled **column-wise**.
- **C language uses row-major order** for storing 2D arrays, while column-major order is commonly associated with some other languages.
- In row-major order, elements of the same row are stored next to each other; in column-major order, elements of the same column are stored next to each other.
- Accessing rows is faster in row-major order, while accessing columns is faster in column-major order.

Example (2D array):

```
int a[2][3] = { {1, 2, 3}, {4, 5, 6} };
```

Row-major memory order:

1 2 3 4 5 6

Column-major memory order:

1 4 2 5 3 6

Conclusion:

Row-major order stores elements row by row and is used in C, whereas column-major order stores elements column by column.

7. How can you pass an array to a function in C? Give an example.

In C, an array is passed to a function by passing the base address of the array. When an array is passed, the function receives a pointer to the first element, so changes made inside the function affect the original array.

According to the notes, the array name itself acts as a **constant pointer** to the first element.

There are two common ways to pass an array to a function:

- By specifying the **array name with size**
 - By using a **pointer parameter**
-

Example: Passing an array to a function

```
#include <stdio.h>
```

```
void display(int a[], int n) {  
    int i;  
    for(i = 0; i < n; i++) {  
        printf("%d ", a[i]);  
    }  
}
```

```
int main() {  
    int arr[5] = {10, 20, 30, 40, 50};  
    display(arr, 5);  
    return 0;  
}
```

Explanation:

- **arr** is passed to the function **display**
- The function receives the **base address of the array**
- Elements are accessed using index notation inside the function

Conclusion:

In C, arrays are passed to functions by reference using the array name, allowing efficient access and modification of array elements.

8. Write a program to count the frequency of each element in an array

```
#include <stdio.h>
```

```
int main() {  
    int a[50], n, i, j, count;  
  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    printf("Enter array elements:\n");  
    for(i = 0; i < n; i++) {
```

```

    scanf("%d", &a[i]);
}

printf("Element  Frequency\n");
for(i = 0; i < n; i++) {
    count = 1;

    if(a[i] == -1)
        continue;

    for(j = i + 1; j < n; j++) {
        if(a[i] == a[j]) {
            count++;
            a[j] = -1; // mark counted element
        }
    }
    printf("%d\t  %d\n", a[i], count);
}

return 0;
}

```

9. Write a program to remove duplicate elements from an array.

```

#include <stdio.h>

int main() {
    int a[50], n, i, j, k;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(a[i] == a[j]) {
                for(k = j; k < n - 1; k++) {

```



```

        a[k] = a[k + 1];
    }
    n--;    // reduce array size
    j--;    // stay at same position
}
}

printf("Array after removing duplicates:\n");
for(i = 0; i < n; i++) {
    printf("%d ", a[i]);
}

return 0;
}

```

10. Write a program to check whether a given matrix is symmetric.

```

#include <stdio.h>

int main() {
    int a[10][10], n, i, j, flag = 1;

    printf("Enter order of the matrix: ");
    scanf("%d", &n);

    printf("Enter matrix elements:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(a[i][j] != a[j][i]) {
                flag = 0;
                break;
            }
        }
    }
    if(flag == 0)

```

```

        break;
    }

    if(flag == 1)
        printf("The matrix is symmetric\n");
    else
        printf("The matrix is not symmetric\n");

    return 0;
}

```

11. Explain how strings are stored using character arrays

In C, a **string is stored as a character array** where each element of the array holds **one character**, and the **last character is a null character ('0')**.

The null character marks the end of the string.

Each character of the string is stored in **contiguous memory locations**, and characters are accessed using **index values starting from 0**.

Example:

```
char s[] = "HELLO";
```

Memory representation:

Index:	0	1	2	3	4	5
Value:	H	E	L	L	O	'0'

- The characters **H, E, L, L, O** are stored sequentially.
- **'0'** is automatically added by the compiler to indicate the end of the string.
- String-handling functions in C identify the string length and end **using the null character**.

Conclusion (Exam Line):

A string in C is stored as a character array with characters placed in contiguous memory locations and terminated by a null character (**'0'**).

12. Write a program to find the number of vowels in a given string using arrays.

```
#include <stdio.h>
```

```

int main() {
    char str[100];
    int i, count = 0;

    printf("Enter a string: ");
    scanf("%s", str);

    for(i = 0; str[i] != '\0'; i++) {
        if(str[i] == 'a' || str[i] == 'e' || str[i] == 'i' ||
           str[i] == 'o' || str[i] == 'u' ||
           str[i] == 'A' || str[i] == 'E' || str[i] == 'I' ||
           str[i] == 'O' || str[i] == 'U') {
            count++;
        }
    }

    printf("Number of vowels = %d\n", count);

    return 0;
}

```

Structure in C

13. What is a structure in C? How is it different from an array?

A **structure in C** is a **user-defined data type** that allows grouping of **related variables of different data types** under a single name. Each variable inside a structure is called a **member**, and members can be of different data types such as **int**, **float**, **char**, arrays, or even other structures.

Example:

```

struct student {
    int roll_no;
    char name[20];
    float marks;
};

```

An **array**, on the other hand, is a collection of **homogeneous elements** (same data type) stored in **contiguous memory locations** and accessed using an index.

Differences between Structure and Array

Structure	Array
Can store different data types	Stores same data type only
Used to represent a record (e.g., student details)	Used to store a list of values
Members are accessed using the dot (.) operator	Elements are accessed using index
Each member has a separate name	All elements share the same name
More suitable for complex data representation	Suitable for simple linear data

Conclusion (Exam Line)

A structure is used to group related data of different data types under one name, whereas an array stores multiple values of the same data type in contiguous memory locations.

This answer is **strictly based on your provided structure notes** and is suitable for a **5-mark question**.

14. Write the syntax for declaring and defining a structure in C

Structure definition:

```
struct structure_name {  
    data_type member1;  
    data_type member2;  
};
```

Structure variable declaration:

```
struct structure_name var1, var2;
```

15. What is the difference between struct and typedef struct?

- **struct** requires the keyword **struct** every time a variable is declared.
- **typedef struct** creates an alias name, so the **struct** keyword is not required while declaring variables.

Example:

```
struct student {  
    int roll;  
};  
struct student s1;    // struct keyword required  
  
typedef struct {  
    int roll;  
} STUDENT;  
STUDENT s1;           // struct keyword not required
```

Conclusion:

typedef struct simplifies structure variable declaration by providing a new data type name.

16. Can a structure contain arrays as members? Give an example.

Yes. A structure in C can contain arrays as its members.

Example:

```
struct student {  
    int roll_no;  
    char name[20]; // array as a member  
    int marks[5];  // array as a member  
};
```

Here, **name** and **marks** are arrays inside the structure.

17. Difference between Structure and Union

Structure	Union
Each member has separate memory location	All members share the same memory location
Total size is sum of sizes of all members	Size is size of the largest member
All members can be used simultaneously	Only one member can be used at a time
Changes in one member do not affect others	Change in one member affects all members
Used when all data members are needed	Used when memory optimization is required

18. Explain how memory is allocated for structure members

- Memory for a **structure is allocated as a single block**.
- Each **member of the structure gets a separate memory location**.
- Structure members are stored **sequentially in memory** in the order of their declaration.
- The **total size of the structure** is approximately the **sum of sizes of all its members**.
- All members can be accessed independently using the **dot (.) operator**.

Conclusion:

In a structure, memory is allocated separately for each member, allowing all members to exist and be used at the same time.

19. What is the purpose of the . (dot) and -> (arrow) operators in structures?

- The **dot (.) operator** is used to access structure members **when a structure variable is used**.
- The **arrow (->) operator** is used to access structure members **when a pointer to a structure is used**.

Example:

```
struct student {
```

```
int roll;
};

struct student s;
struct student *p = &s;

s.roll = 10; // using dot operator
p->roll = 20; // using arrow operator
```

Conclusion:

. is used with structure variables, whereas -> is used with structure pointers.

20. Can we pass a structure to a function in C? If yes, how?

Yes. A structure can be passed to a function in **two ways**:

- **By value:** A copy of the structure is passed (original data not modified).
- **By reference:** A pointer to the structure is passed (original data can be modified).

Example:

```
struct student {
    int roll;
};

void display(struct student s) { // by value
    printf("%d", s.roll);
}

void update(struct student *p) { // by reference
    p->roll = 10;
}
```

Conclusion:

Structures can be passed to functions either by value or by reference using pointers.

21. What are nested structures? Give an example.

Nested structures are structures defined inside another structure. They are used to group **related data logically** within a structure.

Example:

```

struct address {
    char city[20];
    int pin;
};

struct student {
    int roll_no;
    char name[20];
    struct address addr; // nested structure
};

```

Conclusion:

A nested structure allows one structure to be a member of another structure.

22. Can structures contain pointers to themselves? Explain with an example.

Yes. A structure can contain a **pointer to its own type**.

This is used to create **linked data structures** like linked lists.

Example:

```

struct node {
    int data;
    struct node *next; // pointer to same structure
};

```

Conclusion:

Structures can contain self-referential pointers, which allow dynamic data structures such as linked lists to be implemented.

23. Write a program to pass a structure variable to a function and display its contents

```
#include <stdio.h>
```

```

struct student {
    int roll_no;
    char name[20];
};

```

```
void display(struct student s) {
```



```

    printf("Roll No: %d\n", s.roll_no);
    printf("Name: %s\n", s.name);
}

int main() {
    struct student s1;

    printf("Enter roll number: ");
    scanf("%d", &s1.roll_no);
    printf("Enter name: ");
    scanf("%s", s1.name);

    display(s1); // passing structure to function

    return 0;
}

```

Explanation (short):

- The structure variable **s1** is passed **by value** to the function.
- The function accesses members using the **dot (.) operator**.

24. Write a program to find the student with the highest marks using structures

```

#include <stdio.h>

struct student {
    int roll_no;
    char name[20];
    int marks;
};

int main() {
    struct student s[50];
    int n, i, maxIndex = 0;

    printf("Enter number of students: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter roll no, name and marks of student %d:\n", i + 1);
        scanf("%d %s %d", &s[i].roll_no, s[i].name, &s[i].marks);
    }
}

```

```

}

for(i = 1; i < n; i++) {
    if(s[i].marks > s[maxIndex].marks) {
        maxIndex = i;
    }
}

printf("\nStudent with highest marks:\n");
printf("Roll No: %d\n", s[maxIndex].roll_no);
printf("Name: %s\n", s[maxIndex].name);
printf("Marks: %d\n", s[maxIndex].marks);

return 0;
}

```

Explanation (short):

- An **array of structures** is used to store student details.
- The array is traversed to find the **maximum marks**.
- The corresponding student details are displayed.

25. Write a program to calculate the sum of two time values (hours, minutes, seconds) using structures

```
#include <stdio.h>
```

```

struct time {
    int hours;
    int minutes;
    int seconds;
};

```

```

int main() {
    struct time t1, t2, sum;

    printf("Enter first time (hh mm ss): ");
    scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);

    printf("Enter second time (hh mm ss): ");
    scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);

    sum.seconds = t1.seconds + t2.seconds;
}

```

```

sum.minutes = t1.minutes + t2.minutes + sum.seconds / 60;
sum.hours   = t1.hours + t2.hours + sum.minutes / 60;

sum.seconds = sum.seconds % 60;
sum.minutes = sum.minutes % 60;

printf("Sum of time = %d:%d:%d\n", sum.hours, sum.minutes, sum.seconds);

return 0;
}

```

Explanation (short):

- Time values are stored using a **structure**.
- Seconds and minutes are adjusted using **carry-over logic**.
- The final summed time is displayed in **hours:minutes:seconds** format.

You can send the **next question** (26) 👍

26. Define a structure Point with members x and y. Write a program to find the distance between two points

```

#include <stdio.h>
#include <math.h>

struct Point {
    float x;
    float y;
};

int main() {
    struct Point p1, p2;
    float distance;
    printf("Enter coordinates of first point (x y): ");
    scanf("%f %f", &p1.x, &p1.y);
    printf("Enter coordinates of second point (x y): ");
    scanf("%f %f", &p2.x, &p2.y);
    distance = sqrt((p2.x - p1.x)*(p2.x - p1.x) +
                    (p2.y - p1.y)*(p2.y - p1.y));
    printf("Distance between points = %.2f\n", distance);
    return 0;
}

```

Explanation (short):

- A structure **Point** stores **x** and **y** coordinates.
- Distance is calculated using the standard distance formula.

27. Write a program to demonstrate array of structures using employee details

```
#include <stdio.h>

struct employee {
    int emp_id;
    char name[20];
    int salary;
};

int main() {
    struct employee e[50];
    int n, i;

    printf("Enter number of employees: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter employee id, name and salary:\n");
        scanf("%d %s %d", &e[i].emp_id, e[i].name, &e[i].salary);
    }

    printf("\nEmployee Details:\n");
    for(i = 0; i < n; i++) {
        printf("ID: %d Name: %s Salary: %d\n",
            e[i].emp_id, e[i].name, e[i].salary);
    }

    return 0;
}
```

Explanation (short):

- An array of structures is used to store multiple employee records.

- Each structure element holds details of one employee.
- Data is accessed using `array[index].member` format.

28. Write a program to copy one structure variable to another without field-by-field assignment

```
#include <stdio.h>

struct student {
    int roll_no;
    char name[20];
    int marks;
};

int main() {
    struct student s1 = {1, "Ravi", 85};
    struct student s2;

    s2 = s1; // direct structure assignment

    printf("Copied Structure Details:\n");
    printf("Roll No: %d\n", s2.roll_no);
    printf("Name: %s\n", s2.name);
    printf("Marks: %d\n", s2.marks);

    return 0;
}
```

Explanation (short):

- C allows direct assignment of structure variables.
- All members of `s1` are copied to `s2` without field-by-field assignment.

29. Create a structure Date and write a program to check whether two given dates are the same

```
#include <stdio.h>

struct Date {
    int day;
    int month;
    int year;
};
```

```

int main() {
    struct Date d1, d2;

    printf("Enter first date (dd mm yyyy): ");
    scanf("%d %d %d", &d1.day, &d1.month, &d1.year);

    printf("Enter second date (dd mm yyyy): ");
    scanf("%d %d %d", &d2.day, &d2.month, &d2.year);

    if(d1.day == d2.day && d1.month == d2.month && d1.year == d2.year)
        printf("Both dates are the same\n");
    else
        printf("Dates are different\n");

    return 0;
}

```

Explanation (short):

- A structure **Date** stores day, month, and year.
- Corresponding members of both structures are compared to check equality.

30. Explain how to use a pointer to a structure. Write a program to input and display data using structure pointers

Explanation (short):

- A **pointer to a structure** stores the **address of a structure variable**.
- Structure members are accessed using the **arrow (->) operator**.
- It is useful for **efficient function calls and dynamic data handling**.

Program

```
#include <stdio.h>
```

```

struct student {
    int roll_no;
    char name[20];
};

```

```
int main() {
```

```

struct student s;
struct student *p;

p = &s; // pointer to structure

printf("Enter roll number: ");
scanf("%d", &p->roll_no);

printf("Enter name: ");
scanf("%s", p->name);

printf("\nStudent Details:\n");
printf("Roll No: %d\n", p->roll_no);
printf("Name: %s\n", p->name);

return 0;
}

```

Conclusion (exam line):

A structure pointer holds the address of a structure, and its members are accessed using the `->` operator.

31. Create a structure to represent a bank account and write a program to deposit and withdraw money

```

#include <stdio.h>

struct account {
    int acc_no;
    char name[20];
    float balance;
};

int main() {
    struct account a;
    float amt;
    int choice;

    printf("Enter account number, name and balance:\n");
    scanf("%d %s %f", &a.acc_no, a.name, &a.balance);

    printf("1. Deposit\n2. Withdraw\nEnter choice: ");
    scanf("%d", &choice);
}

```

```

if(choice == 1) {
    printf("Enter amount to deposit: ");
    scanf("%f", &amt);
    a.balance += amt;
}
else if(choice == 2) {
    printf("Enter amount to withdraw: ");
    scanf("%f", &amt);
    if(amt <= a.balance)
        a.balance -= amt;
    else
        printf("Insufficient balance\n");
}

printf("\nAccount Details:\n");
printf("Acc No: %d\nName: %s\nBalance: %.2f\n",
    a.acc_no, a.name, a.balance);

return 0;
}

```

Explanation (short):

- A structure stores bank account details.
- Deposit adds amount to balance.
- Withdraw subtracts amount if balance is sufficient.

32. Write a program to sort an array of structures (students by marks)

```
#include <stdio.h>
```

```

struct student {
    int roll_no;
    char name[20];
    int marks;
};

```

```

int main() {
    struct student s[50], temp;
    int n, i, j;
}

```



```

printf("Enter number of students: ");
scanf("%d", &n);

for(i = 0; i < n; i++) {
    printf("Enter roll no, name and marks:\n");
    scanf("%d %s %d", &s[i].roll_no, s[i].name, &s[i].marks);
}

// Sorting by marks (ascending order)
for(i = 0; i < n - 1; i++) {
    for(j = i + 1; j < n; j++) {
        if(s[i].marks > s[j].marks) {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
    }
}

printf("\nStudents sorted by marks:\n");
for(i = 0; i < n; i++) {
    printf("Roll: %d Name: %s Marks: %d\n",
        s[i].roll_no, s[i].name, s[i].marks);
}

return 0;
}

```

Explanation (short):

- Uses an **array of structures**
- Sorting is done by comparing the **marks field**
- Entire structure is swapped using a temporary variable

Abstract Data Types:

33. What is an Abstract Data Type (ADT)? How does it differ from a Data Structure?

An **Abstract Data Type (ADT)** is a **logical or mathematical model** that defines a data type by its **values and operations**, without specifying how it is implemented.

A **Data Structure** is the **physical implementation** of an ADT that shows how data is stored and accessed in memory.

Differences:

- ADT focuses on **what operations are performed**, while a **Data Structure** focuses on **how they are implemented**.
- ADT provides **data abstraction**, whereas a **Data Structure** provides **actual representation**.
- ADT is independent of programming language, while **Data Structure** is language-dependent.
- One ADT can have **multiple data structure implementations**.

Conclusion:

ADT defines the behavior of data, whereas a data structure defines its actual storage and implementation.

34. Why are ADTs important in software development?

- ADTs provide **data abstraction** by hiding implementation details.
- They separate **data definition from operations**, improving clarity.
- ADTs make programs **modular and easier to maintain**.
- They allow **multiple implementations** without changing program logic.
- ADTs help in writing **reusable and reliable code**.

Conclusion:

ADTs improve program design by focusing on behavior rather than implementation.

35. Explain the difference between logical view and implementation view of ADT

- **Logical view of ADT** describes **what the data is and what operations can be performed on it**, without considering how it is implemented.
- **Implementation view of ADT** describes **how the data is stored and how operations are carried out using data structures**.

Difference:

- Logical view focuses on **behavior and operations**.
- Implementation view focuses on **storage and algorithms**.

Conclusion:

The logical view defines *what* an ADT does, while the implementation view defines *how* it is done.

36. List and explain the fundamental operations of the Stack ADT

The **Stack ADT** follows the **LIFO (Last In, First Out)** principle. Its fundamental operations are:

- **Push:** Inserts an element onto the top of the stack.
- **Pop:** Removes the top element from the stack.
- **Peek / Top:** Returns the top element without removing it.
- **isEmpty:** Checks whether the stack is empty.
- **isFull:** Checks whether the stack is full (in array implementation).

Conclusion:

These operations define the behavior of a stack without specifying its implementation.

37. List and explain the fundamental operations of the Queue ADT

The **Queue ADT** follows the **FIFO (First In, First Out)** principle. Its fundamental operations are:

- **Enqueue:** Inserts an element at the **rear** of the queue.
- **Dequeue:** Removes an element from the **front** of the queue.
- **Front / Peek:** Returns the front element without removing it.
- **isEmpty:** Checks whether the queue is empty.
- **isFull:** Checks whether the queue is full (in array implementation).

Conclusion:

These operations define queue behavior independent of how it is implemented.

Pointers in C

38. What is a pointer in C? How is it different from a normal variable?

A **pointer** in C is a variable that **stores the memory address of another variable**.
A **normal variable** stores the **actual value** directly.

Differences:

- A pointer stores an **address**, whereas a normal variable stores a **value**.
- Pointers are declared using the *** operator**; normal variables are not.
- Pointers enable **call by reference**, normal variables support **call by value**.
- Pointers are used for **dynamic memory and data structures**; normal variables are not.
- Pointers allow **direct memory access**, normal variables do not.

Conclusion:

A pointer holds the address of a variable, while a normal variable holds the actual data.

39. What is the use of the * and & operators in pointers?

- The **address operator (&)** is used to **get the memory address** of a variable.
- The **indirection operator (*)** is used to **access the value stored at a memory address** (dereferencing).

Example:

```
int a = 10;  
int *p = &a;
```

```
printf("%d", *p); // prints value of a
```

Conclusion:

& gives the address of a variable, and ***** accesses the value at that address.

40. What is a NULL pointer? Why is it important?

A **NULL pointer** is a pointer that is **not pointing to any valid memory location**. It is usually assigned the value **NULL**.

Importance:

- It indicates that the pointer is **not initialized or not pointing to valid data**.
- Helps in **avoiding accidental access to garbage memory**.
- Used to **check the end of data structures** like linked lists.
- Prevents **runtime errors and crashes**.

Conclusion:

A NULL pointer improves program safety by clearly indicating an invalid or unused pointer.

41. What is a dangling pointer? How can it be avoided?

A **dangling pointer** is a pointer that **points to a memory location that has already been freed or deleted**.

How it occurs:

- When memory is deallocated using `free()` but the pointer is not updated.

How it can be avoided:

- Set the pointer to **NULL after freeing memory**.
- Avoid using pointers after the memory they point to is released.

Example:

```
int *p = (int *)malloc(sizeof(int));  
free(p);  
p = NULL; // avoids dangling pointer
```

Conclusion:

Dangling pointers can cause unpredictable behavior and should be avoided by resetting pointers to NULL after deallocation.

42. Differentiate between call by value and call by reference using pointers

Call by Value	Call by Reference
A copy of the variable is passed to the function	The address of the variable is passed to the function
Changes made inside the function do not affect the original variable	Changes made inside the function affect the original variable
Uses normal variables	Uses pointers
Memory location is different	Memory location is same
Safer but less flexible	More efficient and flexible

Conclusion:

Call by value passes data, while call by reference passes addresses using pointers, allowing modification of original data.

43. Explain pointer arithmetic with an example

Pointer arithmetic refers to performing arithmetic operations on pointers such as **increment, decrement, addition, and subtraction**.

When a pointer is incremented or decremented, it moves by the **size of the data type it points to** (scale factor).

Example:

```
int a[3] = {10, 20, 30};
int *p = a;

printf("%d\n", *p); // 10
p = p + 1;
printf("%d\n", *p); // 20
```

Explanation:

- `p` initially points to `a[0]`
- `p + 1` moves the pointer to the **next integer location**
- Since `int` takes 4 bytes, the pointer advances by 4 bytes

Conclusion:

Pointer arithmetic allows pointers to traverse memory locations based on the data type size.

44. What is the difference between `char *p = "Hello";` and `char arr[] = "Hello";`?

- `char *p = "Hello";`
 - `p` is a **pointer to a string literal**.
 - The string is stored in **read-only memory**.
 - Modifying the string using `p` is **not allowed**.
- `char arr[] = "Hello";`
 - `arr` is a **character array**.
 - The string is stored in **modifiable memory**.
 - The contents of the string **can be modified**.

Conclusion:

`char *p` points to a constant string, while `char arr[]` stores the string in writable memory.

45. Write a program to demonstrate the use of pointers to access and modify variable values

```
#include <stdio.h>

int main() {
    int x = 10;
    int *p;

    p = &x;    // pointer stores address of x

    printf("Value of x = %d\n", x);
    printf("Value using pointer = %d\n", *p);

    *p = 25;    // modify value using pointer

    printf("Modified value of x = %d\n", x);

    return 0;
}
```

Explanation (short):

- Pointer `p` stores the address of `x`.
- `*p` is used to access and modify the value of `x`.
- Changes through the pointer affect the original variable.

46. Write a program to find the length of a string using pointers

```
#include <stdio.h>

int main() {
    char str[50];
    char *p;
    int length = 0;

    printf("Enter a string: ");
    scanf("%s", str);
```

```

p = str; // pointer points to first character

while(*p != '\0') {
    length++;
    p++;
}

printf("Length of string = %d\n", length);

return 0;
}

```

Explanation (short):

- Pointer **p** is used to traverse the string.
- The loop continues until the **null character** (**'\0'**) is reached.
- Each increment of the pointer counts one character.

47. Write a program to copy one string to another using pointer

```

#include <stdio.h>

int main() {
    char str1[50], str2[50];
    char *p1, *p2;

    printf("Enter a string: ");
    scanf("%s", str1);

    p1 = str1;
    p2 = str2;

    while(*p1 != '\0') {
        *p2 = *p1;
        p1++;
        p2++;
    }
    *p2 = '\0'; // terminate string

    printf("Copied string: %s\n", str2);
}

```



```
    return 0;
}
```

Explanation (short):

- Pointers are used to traverse the source string.
- Characters are copied one by one until '\0' is encountered.
- Destination string is null-terminated at the end.

48. Write a program to count the number of vowels in a string using pointers

```
#include <stdio.h>

int main() {
    char str[100];
    char *p;
    int count = 0;

    printf("Enter a string: ");
    scanf("%s", str);

    p = str; // pointer to first character

    while(*p != '\0') {
        if(*p=='a' || *p=='e' || *p=='i' || *p=='o' || *p=='u' ||
            *p=='A' || *p=='E' || *p=='I' || *p=='O' || *p=='U') {
            count++;
        }
        p++;
    }

    printf("Number of vowels = %d\n", count);

    return 0;
}
```

Explanation (short):

- Pointer `p` traverses the string.
- Each character is checked for vowels.
- Count is incremented until the null character is reached.

49. Write a program to find the sum of all elements in an array using pointers

```
#include <stdio.h>

int main() {
    int a[50], n, i, sum = 0;
    int *p;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    p = a; // pointer to first element of array

    for(i = 0; i < n; i++) {
        sum += *p;
        p++;
    }

    printf("Sum of array elements = %d\n", sum);

    return 0;
}
```

Explanation (short):

- Pointer **p** points to the first element of the array.
- Pointer arithmetic is used to traverse the array.
- Each element is added to **sum**.

50. Write a program to reverse a string using pointers

```
#include <stdio.h>

int main() {
    char str[50], temp;
    char *start, *end;
```

```

printf("Enter a string: ");
scanf("%s", str);

start = str;
end = str;

while(*end != '\0') {
    end++;
}
end--; // point to last character

while(start < end) {
    temp = *start;
    *start = *end;
    *end = temp;

    start++;
    end--;
}

printf("Reversed string: %s\n", str);

return 0;
}

```

Explanation (short):

- Two pointers point to the **start and end** of the string.
- Characters are swapped using pointers.
- Pointer movement continues until the string is reversed.

51. Write a program to demonstrate the difference between array of pointers and pointer to array

```

#include <stdio.h>

int main() {
    int a = 10, b = 20, c = 30;

    /* Array of pointers */
    int *arrPtr[3];
    arrPtr[0] = &a;
    arrPtr[1] = &b;

```

```

arrPtr[2] = &c;

/* Pointer to an array */
int arr[3] = {1, 2, 3};
int (*ptrArr)[3] = &arr;

printf("Array of pointers:\n");
printf("%d %d %d\n", *arrPtr[0], *arrPtr[1], *arrPtr[2]);

printf("Pointer to array:\n");
printf("%d %d %d\n", (*ptrArr)[0], (*ptrArr)[1], (*ptrArr)[2]);

return 0;
}

```

Explanation (short):

- **Array of pointers:** Each element is a pointer pointing to different variables.
- **Pointer to array:** A single pointer points to an entire array.

Conclusion:

An array of pointers stores multiple addresses, while a pointer to an array points to the whole array as one unit.

52. Explain the concept of pointers to functions. Write a program to implement the concept

Concept (short):

- A **function pointer** is a pointer that **stores the address of a function**.
- It allows functions to be **passed as arguments**, stored, and called dynamically.
- Syntax depends on the **function's return type and parameters**.

Program: Function Pointer Implementation

```
#include <stdio.h>
```

```

int add(int a, int b) {
    return a + b;
}

```

```
int main() {
```

```

int (*fp)(int, int); // function pointer declaration

fp = add;           // assign function address

printf("Sum = %d\n", fp(10, 20)); // function call using pointer

return 0;
}

```

Explanation (short):

- `fp` stores the address of function `add`
- `fp(10, 20)` calls the function through the pointer

Conclusion:

Function pointers enable dynamic function calls and flexible program design.

53. Write a program to demonstrate pointer to structure (struct *ptr)

```

#include <stdio.h>

struct student {
    int roll_no;
    char name[20];
};

int main() {
    struct student s;
    struct student *ptr;

    ptr = &s; // pointer to structure

    printf("Enter roll number: ");
    scanf("%d", &ptr->roll_no);

    printf("Enter name: ");
    scanf("%s", ptr->name);

    printf("\nStudent Details:\n");
    printf("Roll No: %d\n", ptr->roll_no);
    printf("Name: %s\n", ptr->name);
}

```

```
    return 0;
}
```

Explanation (short):

- **ptr** stores the **address of the structure variable**.
- Structure members are accessed using the **arrow (->) operator**.

54. Write a program to demonstrate passing a structure to a function using pointers

```
#include <stdio.h>
```

```
struct student {
    int roll_no;
    char name[20];
};
```

```
void display(struct student *p) {
    printf("Roll No: %d\n", p->roll_no);
    printf("Name: %s\n", p->name);
}
```

```
int main() {
    struct student s;

    printf("Enter roll number: ");
    scanf("%d", &s.roll_no);

    printf("Enter name: ");
    scanf("%s", s.name);

    display(&s); // passing structure using pointer

    return 0;
}
```

Explanation (short):

- A **pointer to structure** is passed to the function.
- Members are accessed using the **arrow (->) operator**.
- Changes (if any) affect the **original structure**.

55. Write a program to demonstrate array of function pointers

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int sub(int a, int b) {
    return a - b;
}

int mul(int a, int b) {
    return a * b;
}

int main() {
    int (*fp[3])(int, int); // array of function pointers

    fp[0] = add;
    fp[1] = sub;
    fp[2] = mul;

    printf("Addition: %d\n", fp[0](10, 5));
    printf("Subtraction: %d\n", fp[1](10, 5));
    printf("Multiplication: %d\n", fp[2](10, 5));

    return 0;
}
```

Explanation (short):

- **fp** is an array of function pointers.
- Each element points to a different function.
- Functions are called using **fp[index](arguments)**.

Conclusion:

An array of function pointers allows selecting and calling different functions dynamically.

56. Explain the difference between pointer to array and array of pointers with suitable programs

Difference (short and clear)

Pointer to Array	Array of Pointers
Points to an entire array	Contains multiple pointers
Single pointer variable	Collection of pointer variables
Syntax: <code>int (*p)[n]</code>	Syntax: <code>int *p[n]</code>
Used to access array as a whole	Used to point to different variables/arrays

Program 1: Pointer to Array

```
#include <stdio.h>
```

```
int main() {  
    int a[3] = {10, 20, 30};  
    int (*p)[3] = &a;  
  
    printf("%d %d %d\n", (*p)[0], (*p)[1], (*p)[2]);  
  
    return 0;  
}
```

Explanation (short):

- `p` points to the **entire array**
 - Access is done using `(*p)[index]`
-

Program 2: Array of Pointers

```
#include <stdio.h>
```

```
int main() {  
    int x = 10, y = 20, z = 30;  
    int *p[3];  
  
    p[0] = &x;
```



```
p[1] = &y;  
p[2] = &z;  
  
printf("%d %d %d\n", *p[0], *p[1], *p[2]);  
  
return 0;  
}
```

Explanation:

- **p** is an **array of pointers**
- Each pointer points to a **different variable**

UNIT 2

Linked Lists

1. What is a linked list? How does it differ from an array?

A **linked list** is a **linear data structure** in which elements (called **nodes**) are stored in **non-contiguous memory locations**, and each node contains **data** and a **pointer (link)** to the next node.

An **array** is a collection of **homogeneous elements** stored in **contiguous memory locations**.

Differences between Linked List and Array

Linked List	Array
Elements are stored in non-contiguous memory	Elements are stored in contiguous memory
Dynamic size (can grow or shrink)	Fixed size
Insertion and deletion are easy	Insertion and deletion are difficult
Requires extra memory for pointers	No extra memory for pointers
Supports sequential access only	Supports direct (random) access

2. Explain the structure of a node in a singly linked list with an example in C

In a **singly linked list**, each node consists of **two parts**:

- **Info (data) field** – stores the actual data
- **Link (next) field** – stores the address of the next node

The link of the **last node** is **NULL**, indicating the end of the list.

Structure of a node (C example)

```
struct node {  
    int info;  
    struct node *next;  
};
```

Here:

- **info** stores the data
- **next** points to the next node in the list

3. What are the advantages and disadvantages of using a singly linked list?

Advantages

- **Dynamic size** (no fixed memory requirement)
- **Easy insertion and deletion** of nodes
- **Efficient memory utilization** compared to arrays for frequent updates
- **No need for contiguous memory allocation**

Disadvantages

- **Sequential access only** (no direct/random access)
- **Extra memory** required for storing the link (pointer)
- **Traversal is one-directional** (cannot traverse backward)
- **Searching is slower** compared to arrays

4. What is the difference between singly linked list and doubly linked list?

Singly Linked List	Doubly Linked List
Each node has one link (next)	Each node has two links (left and right)
Traversal is only in forward direction	Traversal is both forward and backward
Less memory usage	More memory usage (extra pointer)
Deletion of a node requires previous node pointer	Node can be deleted using its own pointer
Simpler structure	More complex structure

5. How is memory allocated for nodes in a linked list?

- Memory for each node is allocated **dynamically** at runtime.
- Allocation is done using **malloc()**.
- Each node gets memory for **data field** and **link field**.
- Nodes are stored in **non-contiguous memory locations**.
- Memory is released using **free()** when a node is deleted.

6. What are the basic operations performed on a singly linked list?

The basic operations performed on a **singly linked list** are:

- **Insertion** (at beginning, at end, after a given node)
- **Deletion** (from beginning, from end, after a given node)
- **Traversal** (displaying all elements)
- **Searching** (finding a specific element)
- **Updating** (modifying node data)

7. Explain how traversal works in a singly linked list

- Traversal starts from the **first node (head/list pointer)**.
- A temporary pointer is used to visit each node.
- The pointer moves from one node to the next using the **next link**.
- The process continues until the pointer becomes **NULL**.
- During traversal, the **data field of each node is accessed**.

Example logic:

```
p = list;
while(p != NULL) {
    // access p->info
    p = p->next;
}
```

8. Write the structure definition for a singly linked list node in C

```
struct node {
    int info;
    struct node *next;
};
```

- **info** stores the data
- **next** stores the address of the next node

9. Write a program to traverse and display the elements of a singly linked list

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *next;
};

int main() {
    struct node *list = NULL, *temp, *p;
    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        temp = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &temp->info);
        temp->next = NULL;

        if(list == NULL)
            list = temp;
        else {
            p = list;
            while(p->next != NULL)
                p = p->next;
            p->next = temp;
        }
    }

    printf("Linked list elements:\n");
    p = list;
    while(p != NULL) {
        printf("%d ", p->info);
        p = p->next;
    }

    return 0;
}
```

Explanation (short):

- A pointer starts from the **first node**.
- It moves node by node using the **next link**.
- Traversal ends when the pointer becomes **NULL**.

10. Write a function to insert a node at the beginning of a singly linked list

```
void insert_left(int x)
{
    struct node *p;
    p = (struct node *)malloc(sizeof(struct node));
    p->info = x;
    p->next = list;
    list = p;
}
```

- A new node is created.
- The new node points to the current first node.
- The list pointer is updated to the new node.

11. Write a function to insert a node at the end of a singly linked list

```
void insert_last(int x)
{
    struct node *p, *q;
    p = (struct node *)malloc(sizeof(struct node));
    p->info = x;
    p->next = NULL;

    if(list == NULL)
        list = p;
    else {
        q = list;
        while(q->next != NULL)
            q = q->next;
        q->next = p;
    }
}
```

- A new node is created with `next = NULL`.
- If the list is empty, the new node becomes the first node.
- Otherwise, the last node is traversed and linked to the new node.

12. Write a function to insert a node at a given position in a singly linked list

```
void insert_at_position(int x, int pos)
{
    struct node *p, *q;
    int i;
    p = (struct node *)malloc(sizeof(struct node));
    p->info = x;
    if(pos == 1) {        // insert at beginning
        p->next = list;
        list = p;
        return;
    }
    q = list;
    for(i = 1; i < pos - 1 && q != NULL; i++)
        q = q->next;
    if(q == NULL) {
        printf("Invalid position\n");
        free(p);
    } else {
        p->next = q->next;
        q->next = p;
    }
}
```

- Traverses the list up to the given position.
- Inserts the new node by adjusting links.
- Handles insertion at the beginning separately.

13. Write a function to delete the first node from a singly linked list

```
void delete_left()
{
    struct node *p;
    if(list == NULL) {
        printf("Invalid deletion\n");
    } else {
        p = list;
```



```

    list = p->next;
    p->next = NULL;
    free(p);
}
}

```

- Checks if the list is empty
- Moves the list pointer to the next node
- Frees the deleted first node

14. Write a function to delete the last node from a singly linked list

```

void delete_last()
{
    struct node *p, *q;
    if(list == NULL) {
        printf("Invalid deletion\n");
    } else if(list->next == NULL) {
        free(list);
        list = NULL;
    } else {
        q = list;
        while(q->next->next != NULL)
            q = q->next;
        p = q->next;
        q->next = NULL;
        free(p);
    }
}

```

- Handles empty list
- Handles single-node list
- Traverses to the second-last node and deletes the last node

15. Write a function to delete a node at a given position in a singly linked list

```

void delete_at_position(int pos)
{
    struct node *p, *q;
    int i;

    if(list == NULL) {
        printf("Invalid deletion\n");
    }
}

```

```

    return;
}

if(pos == 1) {    // delete first node
    p = list;
    list = p->next;
    p->next = NULL;
    free(p);
    return;
}

q = list;
for(i = 1; i < pos - 1 && q->next != NULL; i++)
    q = q->next;

if(q->next == NULL) {
    printf("Invalid position\n");
} else {
    p = q->next;
    q->next = p->next;
    p->next = NULL;
    free(p);
}
}

```

- Traverses up to the node before the given position
- Adjusts links to remove the target node
- Frees the deleted node

16. Write a program to count the number of nodes in a singly linked list

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *next;
};

int main() {
    struct node *list = NULL, *temp, *p;
    int n, i, count = 0;

```

```

printf("Enter number of nodes: ");
scanf("%d", &n);

for(i = 0; i < n; i++) {
    temp = (struct node *)malloc(sizeof(struct node));
    scanf("%d", &temp->info);
    temp->next = list;
    list = temp;
}

p = list;
while(p != NULL) {
    count++;
    p = p->next;
}

printf("Number of nodes = %d\n", count);

return 0;
}

```

- Traverses the list from the first node
- Increments count for each node
- Stops when pointer becomes **NULL**

17. What is a doubly linked list? How is it different from a singly linked list?

A **doubly linked list** is a linear data structure in which each node contains **three fields**:

data, a pointer to the **previous node**, and a pointer to the **next node**.

Difference between Singly Linked List and Doubly Linked List

Singly Linked List	Doubly Linked List
Node has one link (next)	Node has two links (left and right)
Traversal is only forward	Traversal is forward and backward
Less memory usage	More memory usage

Deletion requires previous node	Node can be deleted directly
Simpler structure	More complex structure

18. Define the structure of a node in a doubly linked list using C

```
struct node {
    int info;
    struct node *left;
    struct node *right;
};
```

- **info** stores the data
- **left** points to the previous node
- **right** points to the next node

19. What are the advantages of a doubly linked list over a singly linked list?

- Allows **traversal in both directions** (forward and backward)
- **Deletion of a node is easier** when the node pointer is given
- No need to track the previous node explicitly
- More efficient for applications requiring **frequent insertions and deletions**
- Suitable for **bidirectional navigation**

20. What are the disadvantages of using a doubly linked list?

- Requires **extra memory** for storing an additional pointer (**left**)
- **More complex structure** compared to singly linked list
- **Insertion and deletion operations** involve updating more pointers
- Slightly **slower operations** due to extra pointer handling
- Not suitable when **space efficiency** is a major concern

21. Explain the role of prev and next pointers in a doubly linked list

- The **prev** (or **left**) pointer stores the address of the **previous node** in the list.
- The **next** (or **right**) pointer stores the address of the **next node** in the list.
- **next** enables **forward traversal** of the list.
- **prev** enables **backward traversal** of the list.

- Together, they allow **easy insertion and deletion** of nodes from both directions.

22. What are the basic operations supported by a doubly linked list?

The basic operations supported by a **doubly linked list** are:

- **Insertion** (at beginning, at end, before or after a given node)
- **Deletion** (from beginning, from end, or a given node)
- **Traversal** (forward and backward)
- **Searching** (finding a specific element)
- **Updating** (modifying node data)

23. Write a program to traverse and display a doubly linked list in forward direction

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
    int info;
    struct node *left;
    struct node *right;
};
```

```
int main() {
    struct node *list = NULL, *temp, *p;
    int n, i;
```

```
    printf("Enter number of nodes: ");
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
        temp = (struct node *)malloc(sizeof(struct node));
        scanf("%d", &temp->info);
        temp->right = NULL;
        temp->left = NULL;
```

```
        if(list == NULL)
            list = temp;
        else {
            p = list;
            while(p->right != NULL)
```

```

        p = p->right;
        p->right = temp;
        temp->left = p;
    }
}

printf("Doubly linked list (forward traversal):\n");
p = list;
while(p != NULL) {
    printf("%d ", p->info);
    p = p->right;
}

return 0;
}

```

- Traversal starts from the **first node**
- Uses the **right (next) pointer** to move forward
- Stops when the pointer becomes **NULL**

24. Write a program to traverse and display a doubly linked list in reverse direction

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *left;
    struct node *right;
};

int main() {
    struct node *list = NULL, *temp, *p;
    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        temp = (struct node *)malloc(sizeof(struct node));
        scanf("%d", &temp->info);
        temp->left = temp->right = NULL;
    }
}

```

```

    if(list == NULL)
        list = temp;
    else {
        p = list;
        while(p->right != NULL)
            p = p->right;
        p->right = temp;
        temp->left = p;
    }
}

/* Move to last node */
p = list;
while(p->right != NULL)
    p = p->right;

printf("Doubly linked list (reverse traversal):\n");
while(p != NULL) {
    printf("%d ", p->info);
    p = p->left;
}

return 0;
}

```

25. Write a function to insert a node at the beginning of a doubly linked list

```

void insert_left(int x)
{
    struct node *p;
    p = (struct node *)malloc(sizeof(struct node));
    p->info = x;
    p->left = NULL;

    if(list == NULL) {
        p->right = NULL;
        list = p;
    } else {
        p->right = list;
        list->left = p;
        list = p;
    }
}

```

```

}
}

```

- A new node is created.
- The new node becomes the **first node**.
- Left link is set to **NULL** and pointers are updated.

26. Write a function to insert a node at the end of a doubly linked list

```

void insert_last(int x)
{
    struct node *p, *temp;

    p = (struct node *)malloc(sizeof(struct node));
    p->info = x;
    p->right = NULL;

    if(list == NULL) {
        p->left = NULL;
        list = p;
    } else {
        temp = list;
        while(temp->right != NULL)
            temp = temp->right;

        temp->right = p;
        p->left = temp;
    }
}

```

- Creates a new node
- Traverses to the last node
- Updates **right** and **left** pointers accordingly

27. Write a function to insert a node at a given position in a doubly linked list

```

void insert_at_position(int x, int pos)
{
    struct node *p, *q;
    int i;

```



```

p = (struct node *)malloc(sizeof(struct node));
p->info = x;

if(pos == 1) { // insert at beginning
    p->left = NULL;
    p->right = list;
    if(list != NULL)
        list->left = p;
    list = p;
    return;
}

q = list;
for(i = 1; i < pos - 1 && q != NULL; i++)
    q = q->right;

if(q == NULL) {
    printf("Invalid position\n");
    free(p);
} else {
    p->right = q->right;
    p->left = q;
    if(q->right != NULL)
        q->right->left = p;
    q->right = p;
}
}

```

- Traverses to the required position
- Adjusts **left** and **right** pointers
- Inserts the new node correctly

28. Write a function to delete the first node of a doubly linked list

```

void delete_left()
{
    struct node *p;

    if(list == NULL) {
        printf("Invalid deletion\n");
    } else {
        p = list;
        list = p->right;
        if(list != NULL)

```

```

        list->left = NULL;
    free(p);
}
}

```

- Checks if the list is empty
- Updates the list pointer to the next node
- Frees the deleted first node

29. Write a function to delete the last node of a doubly linked list

```

void delete_last()
{
    struct node *p;

    if(list == NULL) {
        printf("Invalid deletion\n");
    }
    else if(list->right == NULL) { // only one node
        free(list);
        list = NULL;
    }
    else {
        p = list;
        while(p->right != NULL)
            p = p->right;

        p->left->right = NULL;
        free(p);
    }
}

```

- Traverses to the **last node**
- Updates the previous node's **right** pointer
- Frees the deleted last node

30. Write a function to delete a node at a given position in a doubly linked list

```

void delete_at_position(int pos)
{
    struct node *p;
    int i;

```

```

if(list == NULL) {
    printf("Invalid deletion\n");
    return;
}

if(pos == 1) { // delete first node
    p = list;
    list = p->right;
    if(list != NULL)
        list->left = NULL;
    free(p);
    return;
}

p = list;
for(i = 1; i < pos && p != NULL; i++)
    p = p->right;

if(p == NULL) {
    printf("Invalid position\n");
} else {
    if(p->right != NULL)
        p->right->left = p->left;
    if(p->left != NULL)
        p->left->right = p->right;
    free(p);
}
}

```

- Traverses to the specified position
- Updates both **left** and **right** links
- Frees the deleted node

UNIT 3

Stack

1.Stack – Definition and Primitive Operations (5 Marks)

Definition:

A **stack** is an ordered collection of items in which insertion and deletion are performed **only at one end**, called the **top of the stack**. The last element inserted is the first one to be removed, hence stack follows the **LIFO (Last In First Out)** principle .

Primitive Operations on Stack:

1. PUSH

- Operation used to **insert an element** onto the top of the stack.
- Before pushing, it must be checked whether the stack is full (to avoid overflow).
- On successful push, the **top is incremented** and the element is stored.

2. POP

- Operation used to **remove the top element** from the stack.
- Before popping, it must be checked whether the stack is empty (to avoid underflow).
- On successful pop, the **top is decremented** and the removed element is returned.

These two operations are the **basic (primitive) operations** of a stack .

Below are the **exam-oriented full C programs for Stack implementation using Array and Linked List**, strictly based on your uploaded material.

2. Stack Implementation using Array

```
#include<stdio.h>
#define STACKSIZE 4
#define TRUE 1
#define FALSE 0

struct stack
{
    int top;
    int item[STACKSIZE];
};
```

```

struct stack s;

void push(int);
void pop();
int empty();

void main()
{
    char ch;
    int c, x, i;
    s.top = -1;

    do
    {
        printf("MENU\n");
        printf("1. PUSH \n 2. POP\n 3. Display\n ");
        scanf("%d", &c);

        switch(c)
        {
            case 1:
                printf("enter the no to be inserted\n");
                scanf("%d", &x);
                push(x);
                break;

            case 2:
                pop();
                break;

            case 3:
                printf("elements are\n");
                for(i = s.top; i >= 0; i--)
                {
                    printf("%d ", s.item[i]);
                }
                break;
        }
        printf("\ndo you wish to continue\n");
        scanf(" %c", &ch);
    } while(ch != 'n');
}

void push(int x)

```

```

{
    if(s.top == STACKSIZE - 1)
        printf("stack overflow\n");
    else
        s.item[++s.top] = x;
}

void pop()
{
    if(empty())
        printf("stack underflow\n");
    else
        printf("deleted element is %d\n", s.item[s.top--]);
}

int empty()
{
    if(s.top == -1)
        return TRUE;
    else
        return FALSE;
}

```

3. Stack Implementation using Linked List

```

#include<stdio.h>
#include<stdlib.h>

struct node
{
    int info;
    struct node *next;
};

typedef struct node* nodeptr;

nodeptr list = NULL, temp, p;

nodeptr getnode();
void push(int);
void pop();
void display();

```

```

void main()
{
    int c, x;
    char ch;

    do
    {
        printf("Menu\n");
        printf("1. push \n 2. pop\n 3. display\n");
        scanf("%d", &c);

        switch(c)
        {
            case 1:
                printf("enter the number\n");
                scanf("%d", &x);
                push(x);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;
        }
        printf("\ndo you want to continue\n");
        scanf(" %c", &ch);
    } while(ch != 'n');
}

```

```

nodeptr getnode()
{
    nodeptr p;
    p = (nodeptr)malloc(sizeof(struct node));
    return p;
}

```

```

void push(int x)
{
    temp = getnode();
    temp->info = x;
}

```



```

temp->next = list;
list = temp;
}

void pop()
{
    if(list == NULL)
        printf("deletion not possible. No nodes existed\n");
    else
    {
        p = list;
        list = p->next;
        printf("Deleted node info value is %d\n", p->info);
        free(p);
    }
}

void display()
{
    if(list == NULL)
        printf("No nodes in the list\n");

    p = list;
    while(p != NULL)
    {
        printf("%d\t", p->info);
        p = p->next;
    }
}

```

Below is a **well-structured exam-ready answer** exactly as you asked.

4.Applications of Stack

A **stack** is widely used in computer science due to its **LIFO (Last In First Out)** nature. Important applications of stack include:

1. Evaluating Postfix Expressions

- Operands are pushed onto the stack.
- When an operator is encountered, operands are popped, the operation is performed, and the result is pushed back.

2. **Conversion of Infix Expression to Postfix Expression**
 - Operators and parentheses are managed using a stack based on precedence rules.
3. **Checking Balanced Parentheses**
 - Stack is used to match opening and closing brackets like (), {}, [].
4. **Function Calls and Recursion**
 - Function calls and local variables are stored in stack memory.
5. **Reversal of Data**
 - Stack can reverse strings, numbers, and arrays.

(Stacks are especially useful where **temporary storage and backtracking** are required.)

5. Evaluating a Postfix Expression

Explanation

- Scan the postfix expression from **left to right**.
- If the symbol is an **operand**, push it onto the stack.
- If the symbol is an **operator**:
 - Pop two operands.
 - Perform the operation.
 - Push the result back onto the stack.
- After scanning the entire expression, the **remaining element in the stack is the result**.

This logic directly follows the algorithm provided in your notes .

C Program: Postfix Expression Evaluation

```
#include<stdio.h>
#include<math.h>
#define STACKSIZE 100
#define TRUE 1
#define FALSE 0

struct stack
{
    int top;
    int item[STACKSIZE];
};
```

```

struct stack s;

void push(int);
int pop();
int evaluate(char []);
int empty();
int operation(int, int, char);

void main()
{
    char postfix[100];
    int value;
    s.top = -1;

    printf("enter postfix string \n");
    scanf("%s", postfix);

    value = evaluate(postfix);

    if(empty())
        printf("The evaluated answer = %d", value);
    else
        printf("Invalid expression\n");
}

int evaluate(char postfix[])
{
    int i;
    char symb;
    int opnd1, opnd2, value;

    for(i = 0; postfix[i] != '\0'; i++)
    {
        symb = postfix[i];
        if(symb >= '0' && symb <= '9')
            push(symb - '0');
        else
        {
            opnd2 = pop();
            opnd1 = pop();
            value = operation(opnd1, opnd2, symb);
            push(value);
        }
    }
}

```

```

    }
    return pop();
}

int operation(int opnd1, int opnd2, char symb)
{
    switch(symb)
    {
        case '+': return opnd1 + opnd2;
        case '-': return opnd1 - opnd2;
        case '*': return opnd1 * opnd2;
        case '/': return opnd1 / opnd2;
        case '$': return pow(opnd1, opnd2);
    }
}

void push(int x)
{
    s.item[++s.top] = x;
}

int pop()
{
    return s.item[s.top--];
}

int empty()
{
    return (s.top == -1);
}

```

6. Converting Infix Expression to Postfix

Explanation

- Scan the infix expression from **left to right**.
- If the symbol is an **operand**, add it directly to postfix.
- If the symbol is:
 - '(' → push onto stack
 - ')' → pop until '(' is found

- Operator → pop operators from stack with **higher or equal precedence**, then push current operator
- After scanning, pop all remaining operators from the stack.

This follows the standard algorithm given in your notes .

C Program: Infix to Postfix Conversion

```
#include<stdio.h>
#define STACKSIZE 100
#define TRUE 1
#define FALSE 0

struct stack
{
    int top;
    char item[STACKSIZE];
};

struct stack s;
char infix[100], postfix[100];
int pos = 0;

void push(char);
char pop();
void convert();
int empty();
int precedence(char);

void main()
{
    s.top = -1;
    printf("enter infix string \n");
    scanf("%s", infix);

    convert();

    printf("The postfix expression:\n");
    printf("%s", postfix);
}

void convert()
{
```

```

int i;
char symb, temp;

for(i = 0; infix[i] != '\0'; i++)
{
    symb = infix[i];

    switch(symb)
    {
        case '(':
            push(symb);
            break;

        case ')':
            while((temp = pop()) != '(')
                postfix[pos++] = temp;
            break;

        case '+':
        case '-':
        case '*':
        case '/':
        case '$':
            while(!empty() && precedence(s.item[s.top]) >= precedence(symb))
            {
                temp = pop();
                postfix[pos++] = temp;
            }
            push(symb);
            break;

        default:
            postfix[pos++] = symb;
    }
}

while(!empty())
    postfix[pos++] = pop();
}

void push(char x)
{
    s.item[++s.top] = x;
}

```

```

char pop()
{
    return s.item[s.top--];
}

int empty()
{
    return (s.top == -1);
}

int precedence(char symb)
{
    switch(symb)
    {
        case '$': return 3;
        case '*':
        case '/': return 2;
        case '+':
        case '-': return 1;
        default : return 0;
    }
}

```

7.Recursion: Definition and Process (5 Marks)

Definition of Recursion

Recursion is the ability of a function to **call itself** until a specified **stopping (terminating) condition** is satisfied.

Every recursive function **must contain a stopping condition**; otherwise, it will result in an **infinite loop** .

A definition that expresses a problem in terms of a **simpler case of itself** is called a **recursive definition** .

Recursive Process

- When a recursive function is called, it **does not execute immediately**.
- Each function call is **pushed onto the system stack**.
- This continues until the **stopping condition** is reached.

- After reaching the stopping condition, function calls are **popped from the stack in reverse order** and executed one by one .
-

Example (Factorial – Conceptual Explanation)

- Factorial is defined as:
 - ($n! = 1$), if ($n = 0$)
 - ($n! = n \times (n-1)!$), if ($n > 0$)
 - Here, the function keeps calling itself with a smaller value until ($n = 0$), which acts as the **stopping condition** .
-

Important Points

- Recursive calls use **stack memory**.
- Recursive solutions are often **simple and natural** for problems like factorial, Fibonacci, and Towers of Hanoi.
- Recursive programs are generally **less efficient than iterative programs** in terms of time and space due to stack usage .

8.Implementation of Recursion Programs in C

Recursion is implemented by writing a function that calls itself with a base (stopping) condition. The recursive calls are managed using the system stack

9. Recursive Program for Factorial

Explanation

- Base condition: $n == 0$
- Recursive case: $n * \text{fact}(n-1)$

```
#include<stdio.h>
```

```
int fact(int n)
{
    if(n == 0)
        return 1;
    else
        return n * fact(n - 1);
}
```



```

void main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("Factorial = %d", fact(n));
}

```

10. Recursive Program for Fibonacci Series

Explanation

Base condition: $n == 0$ or $n == 1$

Recursive case: $\text{fib}(n-1) + \text{fib}(n-2)$

```
#include<stdio.h>
```

```

int fib(int n)
{
    if(n == 0 || n == 1)
        return n;
    else
        return fib(n - 1) + fib(n - 2);
}

```

```

void main()
{
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        printf("%d ", fib(i));
}

```

11. Recursive Program for Greatest Common Divisor (GCD)

Explanation

- Base condition: `b == 0`
- Recursive case: `gcd(b, a % b)`

```
#include<stdio.h>
int gcd(int a, int b)
{
    if(b == 0)
        return a;
    else
        return gcd(b, a % b);
}
void main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
    printf("GCD = %d", gcd(a, b));
}
```

12. Recursive Program for Multiplication of Two Numbers

Explanation

- Multiplication is defined as repeated addition
- Base condition: `b == 1`
- Recursive case: `mult(a, b-1) + a`

```
#include<stdio.h>

int mult(int a, int b)
{
    if(b == 1)
        return a;
    else
        return mult(a, b - 1) + a;
}

void main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
}
```

```
printf("Multiplication = %d", mult(a, b));  
}
```

13. Recursive Program for Towers of Hanoi

Explanation

- Move **n-1** disks from source to auxiliary
- Move nth disk to destination
- Move **n-1** disks from auxiliary to destination

```
#include<stdio.h>
```

```
void towers(int n, char src, char dst, char aux)  
{  
    if(n == 1)  
        printf("Move disk 1 from %c to %c\n", src, dst);  
    else  
    {  
        towers(n - 1, src, aux, dst);  
        printf("Move disk %d from %c to %c\n", n, src, dst);  
        towers(n - 1, aux, dst, src);  
    }  
}
```

```
void main()  
{  
    int n;  
    printf("Enter number of disks: ");  
    scanf("%d", &n);  
    towers(n, 'A', 'C', 'B');  
}
```

Queue

14.Queue – Definition and Primitive Operations (5 Marks)

Definition of Queue

A queue is an ordered collection of elements in which insertion is done at one end called the *rear* and deletion is done at the other end called the *front*. Queue follows the principle of FIFO (First In First Out), that is, the element inserted first is removed first .

Primitive Operations on Queue

1. INSERT (Enqueue)
 - Operation used to add an element at the rear end of the queue.
 - Before insertion, it must be checked whether the queue is full to avoid overflow.
2. DELETE (Dequeue)
 - Operation used to remove an element from the front end of the queue.
 - Before deletion, it must be checked whether the queue is empty to avoid underflow.
3. EMPTY
 - Checks whether the queue contains any elements.
 - If $\text{rear} < \text{front}$, the queue is considered empty.

Important Points

- Queue uses two pointers: *front* and *rear*.
- Overflow occurs when no more elements can be inserted.
- Underflow occurs when deletion is attempted on an empty queue.

15. Ordinary Queue using Array

```
#include<stdio.h>
#define QSIZE 4
#define TRUE 1
#define FALSE 0
```

```
struct Queue
{
    int rear, front;
    int item[QSIZE];
};
```

```

struct Queue q;

void insert(int);
int delete();
int empty();

void main()
{
    char ch;
    int c, x, i;
    q.rear = -1;
    q.front = 0;

    do
    {
        printf("MENU\n");
        printf("1. INSERT \n 2. DELETE\n 3. Display\n ");
        scanf("%d", &c);

        switch(c)
        {
            case 1:
                printf("enter the no to be inserted\n");
                scanf("%d", &x);
                insert(x);
                break;

            case 2:
                x = delete();
                if(x != -1)
                    printf("deleted element is %d\n", x);
                break;

            case 3:
                printf("elements are\n");
                for(i = q.rear; i >= q.front; i--)
                    printf("%d ", q.item[i]);
                break;
        }
        printf("\ndo you wish to continue\n");
        scanf(" %c", &ch);
    } while(ch != 'n');
}

```

```

void insert(int x)
{
    if(q.rear == QSIZE - 1)
        printf("Queue overflow\n");
    else
        q.item[++q.rear] = x;
}

int delete()
{
    if(empty())
    {
        printf("Queue underflow\n");
        return -1;
    }
    return q.item[q.front++];
}

int empty()
{
    if(q.rear < q.front)
        return TRUE;
    else
        return FALSE;
}

```

16. Ordinary Queue using Linked List

```

#include<stdio.h>
#include<stdlib.h>

struct node
{
    int info;
    struct node *next;
};

typedef struct node* NODEPTR;
NODEPTR front = NULL, rear = NULL;

NODEPTR getnode();
void insert(int);

```

```

int delete();
void display();

void main()
{
    int c, x;
    char ch;

    do
    {
        printf("MENU\n");
        printf("1. INSERT \n 2. DELETE\n 3. DISPLAY\n");
        scanf("%d", &c);

        switch(c)
        {
            case 1:
                printf("Enter element\n");
                scanf("%d", &x);
                insert(x);
                break;

            case 2:
                x = delete();
                if(x != -1)
                    printf("Deleted element = %d\n", x);
                break;

            case 3:
                display();
                break;
        }
        printf("\nDo you wish to continue\n");
        scanf(" %c", &ch);
    } while(ch != 'n');
}

NODEPTR getnode()
{
    NODEPTR p;
    p = (NODEPTR)malloc(sizeof(struct node));
    return p;
}

```

```

void insert(int x)
{
    NODEPTR p = getnode();
    p->info = x;
    p->next = NULL;

    if(rear == NULL)
        front = p;
    else
        rear->next = p;

    rear = p;
}

int delete()
{
    NODEPTR p;
    int x;

    if(front == NULL)
    {
        printf("Queue underflow\n");
        return -1;
    }

    p = front;
    x = p->info;
    front = front->next;

    if(front == NULL)
        rear = NULL;

    free(p);
    return x;
}

void display()
{
    NODEPTR p = front;
    if(p == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

```



```

printf("Queue elements: ");
while(p != NULL)
{
    printf("%d ", p->info);
    p = p->next;
}
}

```

17. Circular Queue using Array

```

#include<stdio.h>
#define maxsize 4
#define TRUE 1
#define FALSE 0

struct CQueue
{
    int rear, front;
    int item[maxsize];
};

struct CQueue cq;

void insert(int);
int delete();
int empty();

void main()
{
    char ch;
    int c, x, i;
    cq.rear = maxsize - 1;
    cq.front = maxsize - 1;

    do
    {
        printf("MENU\n");
        printf("1. INSERT \n 2. DELETE\n 3. Display\n ");
        scanf("%d", &c);

        switch(c)

```

```

{
    case 1:
        printf("enter the no to be inserted\n");
        scanf("%d", &x);
        insert(x);
        break;

    case 2:
        x = delete();
        if(x == -1)
            printf("Queue underflow\n");
        else
            printf("deleted element is %d\n", x);
        break;

    case 3:
        printf("elements are\n");
        if(cq.front == maxsize - 1)
        {
            for(i = 0; i <= cq.rear; i++)
                printf("%d ", cq.item[i]);
        }
        else
        {
            for(i = cq.front + 1; i <= cq.rear; i++)
                printf("%d ", cq.item[i]);
        }
        break;
    }
    printf("\ndo you wish to continue\n");
    scanf(" %c", &ch);
} while(ch != 'n');
}

```

```

void insert(int x)
{
    if(cq.front == (cq.rear + 1) % maxsize)
        printf("Queue overflow\n");
    else
    {
        cq.rear = (cq.rear + 1) % maxsize;
        cq.item[cq.rear] = x;
    }
}

```

```

int delete()
{
    if(empty())
        return -1;
    else
    {
        cq.front = (cq.front + 1) % maxsize;
        return cq.item[cq.front];
    }
}

```

```

int empty()
{
    if(cq.rear == cq.front)
        return TRUE;
    else
        return FALSE;
}

```

18. Circular Queue using Linked List

Note (as per syllabus focus):

Circular queue is commonly implemented using **arrays** in your notes.

When implemented using linked lists, the **last node points back to the first node**, forming a circle.

```

#include<stdio.h>
#include<stdlib.h>

```

```

struct node
{
    int info;
    struct node *next;
};

```

```

typedef struct node* NODEPTR;
NODEPTR rear = NULL;

```

```

void insert(int);
int delete();
void display();

```

```

void main()
{
    int c, x;
    char ch;

    do
    {
        printf("MENU\n");
        printf("1. INSERT \n 2. DELETE\n 3. DISPLAY\n");
        scanf("%d", &c);

        switch(c)
        {
            case 1:
                printf("Enter element\n");
                scanf("%d", &x);
                insert(x);
                break;

            case 2:
                x = delete();
                if(x != -1)
                    printf("Deleted element = %d\n", x);
                break;

            case 3:
                display();
                break;
        }
        printf("\nDo you wish to continue\n");
        scanf(" %c", &ch);
    } while(ch != 'n');
}

```

```

void insert(int x)
{
    NODEPTR p = (NODEPTR)malloc(sizeof(struct node));
    p->info = x;

    if(rear == NULL)
    {
        rear = p;
        rear->next = rear;
    }
}

```

```

    else
    {
        p->next = rear->next;
        rear->next = p;
        rear = p;
    }
}

int delete()
{
    NODEPTR p;
    int x;

    if(rear == NULL)
    {
        printf("Queue underflow\n");
        return -1;
    }

    p = rear->next;
    x = p->info;

    if(p == rear)
        rear = NULL;
    else
        rear->next = p->next;

    free(p);
    return x;
}

void display()
{
    NODEPTR p;
    if(rear == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    p = rear->next;
    do
    {
        printf("%d ", p->info);

```

```
    p = p->next;  
} while(p != rear->next);  
}
```

UNIT 4

Binary Trees

1.Binary Trees – Basic Terminology (5 Marks)

A **binary tree** is a finite set of elements which is either **empty** or consists of a **root node** and two disjoint binary trees called the **left subtree** and **right subtree**. Each element in a binary tree is called a **node** .

Basic Terminologies of Binary Tree

1. **Root**
 - The topmost node of a binary tree is called the **root**.
2. **Parent and Child**
 - If a node has left or right subtrees, it is called the **parent**, and the roots of those subtrees are its **children**.
3. **Leaf Node**
 - A node with **no children** is called a **leaf**.
4. **Ancestor and Descendant**
 - A node $N1$ is an **ancestor** of node $N2$ if $N2$ lies in the subtree rooted at $N1$.
 - $N2$ is a **descendant** of $N1$.
5. **Left and Right Descendants**
 - Nodes in the left subtree are **left descendants**.
 - Nodes in the right subtree are **right descendants**.
6. **Sibling (Brother Nodes)**
 - Two nodes having the **same parent** are called siblings.
7. **Level of a Node**
 - The **root is at level 0**.
 - The level of any other node is one more than its parent's level.
8. **Depth of a Binary Tree**
 - The **maximum level of any leaf node** is called the depth of the tree.
9. **Strictly Binary Tree**
 - A binary tree in which **every non-leaf node has exactly two children**.
10. **Complete Binary Tree**
 - A strictly binary tree in which **all leaf nodes are at the same level**.

2.Operations on Binary Trees (5 Marks)

The **basic (primitive) operations** that can be performed on a binary tree are listed below. These operations are defined with respect to a node p , which is a pointer to a node in the binary tree .

1. info(p)

- Returns the **information (data)** stored in the node pointed to by p .

2. left(p)

- Returns a pointer to the **left child** of the node p .
- If the node has no left child, it returns **NULL**.

3. right(p)

- Returns a pointer to the **right child** of the node p .
- If the node has no right child, it returns **NULL**.

4. Creation of a Node

- A new node is created with given data and its left and right child pointers are initialized to **NULL**.

5. Insertion of Nodes

- Nodes are inserted as **left or right children** of a given node using operations like **setleft(p, x)** and **setright(p, x)**.

Important Note

- Using these primitive operations, complex operations such as **traversal, searching, and insertion** can be performed on binary trees.
- These operations form the **foundation for Binary Search Trees and other tree structures**.

3.Applications of Binary Trees (5 Marks)

A **binary tree** is a useful data structure when **two-way decisions** must be made at each step of a process. Important applications are:

1. **Searching and Duplicate Detection**
 - Binary trees are used to **detect duplicate elements** efficiently.
 - Each new element is compared with the root; if smaller it goes to the left subtree, if larger to the right subtree.
 - If an element matches an existing node, it is identified as a **duplicate**.
2. **Binary Search Trees (BSTs)**
 - Used to **store data in sorted order**.
 - Enable efficient **searching, insertion, and deletion** based on the BST property ($\text{left} < \text{root} \leq \text{right}$).

3. Expression Representation and Evaluation

- Binary trees are used to represent **arithmetic expressions**, where internal nodes are operators and leaf nodes are operands.

4. Tree Traversals

- Inorder, preorder, and postorder traversals are used to **process hierarchical data**, printing or accessing nodes in different orders.

5. Decision Making Processes

- Binary trees model **decision trees**, where each node represents a decision with two possible outcomes.

4.Binary Tree Representation: Node Representation of Binary Tree (5 Marks)

In **node representation of a binary tree**, each node of the tree is represented as a **structure** containing information and links to its related nodes. This representation can be implemented using **arrays or dynamic memory allocation** .

Structure of a Node

Each node contains the following fields:

- **info** – stores the data of the node
- **left** – pointer/index to the left child
- **right** – pointer/index to the right child
- **parent** – pointer/index to the parent node (optional)

Array-based Node Representation

```
#define NUMNODES 500
```

```
struct nodetype
```

```
{
```

```
    int info;
```

```
    int left;
```

```
    int right;
```

```
    int parent;
```

```
};
```

```
struct nodetype node[NUMNODES];
```

- Nodes are stored in an array.
- **left**, **right**, and **parent** store **indices** of related nodes.
- Also called **linked array representation**.
- Nodes are taken from an available pool and returned when not in use .

Dynamic Node Representation

struct node

{

int info;

struct node *left;

struct node *right;

struct node *parent;

};

- Uses **dynamic memory allocation**.
- **left** and **right** point directly to child nodes.
- **parent** pointer may be omitted if traversal is only top-down.

Advantages

- Efficient representation of hierarchical data.
- Easy insertion and deletion of nodes.
- Suitable for non-complete binary trees.

5.Array Representation of Binary Trees (5 Marks)

In **array representation of a binary tree**, the nodes of the tree are stored **sequentially in an array** instead of using pointers. This method is also called **implicit array representation** or **sequential representation** of a binary tree .

Concept

- The **root node** of the binary tree is stored at **index 0**.
- For a node stored at index **p**:
 - **Left child** is stored at index **$2p + 1$**

- **Right child** is stored at index $2p + 2$
- This representation is most suitable for **almost complete binary trees**.

Example

If a binary tree is stored in an array starting from index 0:

Index	0	1	2	3	4	5	6
Data	A	B	C	D	E	F	G

- - A is the root
- B and C are left and right children of A
- D and E are children of B
- F and G are children of C

Characteristics

- No explicit **left, right, or parent pointers** are required.
- Memory is used efficiently only when the tree is **almost complete**.
- Wastage of memory occurs for **skewed or sparse trees**.

Advantages

- Simple implementation.
- Faster access using index calculation.

Disadvantage

- Not suitable for general binary trees with many missing nodes.

6.Binary Tree Traversals and Related Properties

Binary Tree Traversal

Traversal of a binary tree means **visiting every node of the tree exactly once** and processing (printing) the data in a specific order. Since there is no natural linear order in trees, different traversal methods are used .

Types of Binary Tree Traversals

1. Preorder Traversal

- Visit the **root**
- Traverse the **left subtree** in preorder
- Traverse the **right subtree** in preorder

Order: Root → Left → Right

Used to **copy a tree** and to get **prefix expressions**.

2. Inorder Traversal

- Traverse the **left subtree** in inorder
- Visit the **root**
- Traverse the **right subtree** in inorder

Order: Left → Root → Right

In a **Binary Search Tree**, inorder traversal gives elements in **sorted order**.

3. Postorder Traversal

- Traverse the **left subtree** in postorder
- Traverse the **right subtree** in postorder
- Visit the **root**

Order: Left → Right → Root

Used to **delete a tree** and to get **postfix expressions**.

Related Properties of Binary Tree

- Each traversal visits **all nodes exactly once**.
- Traversal order depends on the **position of root visit**.
- Inorder traversal of a **BST** produces a **sorted sequence**.
- Preorder and postorder traversals are useful for **expression trees**.
- Traversals can be implemented using **recursion or stack**.

7.Threaded Binary Trees (5 Marks)

A **threaded binary tree** is a binary tree in which the **NULL pointers** (left or right) are replaced by **threads** that point to the node's **inorder predecessor or inorder successor**. This allows traversal of the tree **without using recursion or stack**.

Need for Threaded Binary Trees

- In normal binary trees, many left and right pointers are **NULL**.
 - During **non-recursive inorder traversal**, a **stack** is required.
 - By converting NULL pointers into threads, traversal can be done **efficiently without stack**.
-

Types of Threaded Binary Trees

1. **Right-in Threaded Binary Tree**
 - The **right NULL pointer** of a node points to its **inorder successor**.
 - A logical field **rthread** is used:
 - **rthread = TRUE** → right pointer is a thread
 - **rthread = FALSE** → right pointer is a normal right child
 2. **Left-in Threaded Binary Tree**
 - The **left NULL pointer** of a node points to its **inorder predecessor**.
 3. **Fully (In) Threaded Binary Tree**
 - Both left and right NULL pointers are replaced by threads to inorder predecessor and successor respectively.
-

Structure of a Node (Right-in Threaded Tree)

```
struct node
{
    int info;

    struct node *left;

    struct node *right;

    int rthread;
};
```

Advantages

- Inorder traversal **without recursion or stack**
- Efficient use of NULL pointers
- Faster traversal compared to normal binary trees

Graphs

8. Graphs – Definition (5 Marks)

A **graph** is a **non-linear data structure** consisting of a **finite set of vertices (nodes)** and a set of **edges** that connect pairs of vertices.

Formally, a graph is defined as

$$G = (V, E)$$

where:

- **V** is a finite, non-empty set of **vertices**
 - **E** is a set of **edges**, where each edge connects two vertices in **V**
-

Basic Terms in Graphs

1. **Vertex (Node)**
 - A fundamental unit of a graph representing an object.
 2. **Edge**
 - A connection between two vertices.
 3. **Directed Graph (Digraph)**
 - A graph in which edges have a **direction** ($u \rightarrow v$).
 4. **Undirected Graph**
 - A graph in which edges have **no direction** ($u - v$).
 5. **Weighted Graph**
 - A graph in which edges have **weights or costs** associated with them.
-

Key Points

- Graphs are used to represent **networks** such as computer networks, road maps, and social networks.
- Unlike trees, graphs **may contain cycles** and **may not be connected**.

9. Graph Representation: Adjacency Matrix and Adjacency List (5 Marks)

Graphs can be represented in memory mainly using **Adjacency Matrix** and **Adjacency List** methods.

1. Adjacency Matrix Representation

- An **adjacency matrix** is a 2-dimensional array of size $n \times n$, where n is the number of vertices.
- If there is an edge between vertex i and vertex j , then $\text{matrix}[i][j] = 1$ (or weight, in case of weighted graph), otherwise 0.

Example

For a graph with vertices A, B, C:

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

Characteristics

- Simple and easy to implement
 - Requires $O(n^2)$ memory
 - Suitable for **dense graphs**
-

2. Adjacency List Representation

- An **adjacency list** represents a graph as an array of linked lists.
- Each vertex has a list containing all vertices **adjacent to it**.

Example

- $A \rightarrow B$
- $B \rightarrow A \rightarrow C$
- $C \rightarrow B$

Characteristics

- Requires **less memory**
 - Memory usage is $O(n + e)$ where e is number of edges
 - Suitable for **sparse graphs**
-

Comparison (Key Points)

Adjacency Matrix	Adjacency List
Uses more memory	Uses less memory
Fast edge lookup	Slower edge lookup
Simple structure	Efficient for sparse graphs

10.Graph Traversal Techniques: BFS and DFS (5 Marks)

Graph traversal means **visiting all the vertices of a graph exactly once** in a systematic manner. The two commonly used traversal techniques are **Breadth First Search (BFS)** and **Depth First Search (DFS)**.

1. Breadth First Search (BFS)

- **BFS traverses the graph level by level.**
- It starts from a given source vertex and visits **all its adjacent vertices first**, before moving to the next level.
- BFS uses a **Queue** data structure.

Procedure (Idea)

1. Start from a source vertex and mark it as visited.
2. Insert it into a queue.
3. Remove a vertex from the queue and visit all its unvisited adjacent vertices.
4. Insert the adjacent vertices into the queue.
5. Repeat until the queue becomes empty.

Applications

- Finding **shortest path** in unweighted graphs
 - Level-wise traversal
 - Checking graph connectivity
-

2. Depth First Search (DFS)

- DFS traverses the graph **depth-wise**.
- It explores as far as possible along one branch before backtracking.
- DFS uses **Stack** (explicitly) or **Recursion** (implicitly).

Procedure (Idea)

1. Start from a source vertex and mark it as visited.
2. Visit an unvisited adjacent vertex.
3. Continue the process recursively until no unvisited adjacent vertex is left.
4. Backtrack and repeat for other vertices.

Applications

- Detecting **cycles** in a graph
 - Finding **connected components**
 - Topological sorting (in directed acyclic graphs)
-

Key Differences

BFS	DFS
Uses Queue	Uses Stack / Recursion

Level-wise traversal	Depth-wise traversal
Finds shortest path	Does not guarantee shortest path
More memory	Less memory (generally)

11.Applications of Graphs (5 Marks)

A **graph** is used to represent **relationships between objects**. Due to this property, graphs are widely used in many real-world applications.

Important Applications of Graphs

1. Computer Networks

- Computers are represented as **vertices** and communication links as **edges**.
- Used in **routing algorithms** and network topology design.

2. Transportation Systems

- Cities or locations are vertices and roads, railways, or air routes are edges.
- Used to find **shortest paths** and optimal routes.

3. Social Networks

- Users are vertices and relationships (friends/followers) are edges.
- Used in **recommendation systems** and connection analysis.

4. Web Page Linking (WWW)

- Web pages are vertices and hyperlinks are edges.
- Used by search engines for **page ranking**.

5. Scheduling and Project Planning

- Tasks are vertices and dependencies are edges.
- Used in **PERT/CPM** charts and job scheduling.

6.Greedy Technique,

7.Prim's Algorithm,

8.Kruskal's Algorithm,

9.Dijkstra's Algorithm.

UNIT 5

Sorting- Exchange sort

1. Bubble Sort (Exchange Sort)

Idea:

Adjacent elements are compared and **exchanged** if they are in the wrong order. After each pass, the **largest element bubbles to the end**.

```
#include<stdio.h>
```

```
void bubble_sort(int a[], int n)
```

```
{
    int i, j, temp;
    for(i = 0; i < n-1; i++)
    {
        for(j = 0; j < n-1-i; j++)
        {
            if(a[j] > a[j+1])
            {
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
}
```

```
void main()
```

```
{
    int a[20], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    bubble_sort(a, n);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
}
```

2. Selection Sort (Exchange Sort)

Idea:

In each pass, the **largest element** from the unsorted part is selected and **exchanged** with the element at the end.

```
#include<stdio.h>
```

```
void selection_sort(int a[], int n)
{
    int i, j, large, index;
    for(i = n-1; i > 0; i--)
    {
        large = a[0];
        index = 0;
        for(j = 1; j <= i; j++)
        {
            if(a[j] > large)
            {
                large = a[j];
                index = j;
            }
        }
        a[index] = a[i];
        a[i] = large;
    }
}
```

```
void main()
{
    int a[20], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    selection_sort(a, n);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
```

```
    printf("%d ", a[i]);  
}
```

3. Quick Sort (Exchange Sort)

Idea:

Quick sort selects a **pivot**, partitions the array into elements **less than** and **greater than** the pivot, and recursively sorts the subarrays.

```
#include<stdio.h>
```

```
int partition(int x[], int low, int high)
```

```
{  
    int a, i, j, temp;  
    a = x[low];  
    i = low + 1;  
    j = high;  
  
    while(i < j)  
    {  
        while(x[i] <= a && i < high)  
            i++;  
        while(x[j] > a && j > low)  
            j--;  
        if(i < j)  
        {  
            temp = x[i];  
            x[i] = x[j];  
            x[j] = temp;  
        }  
    }  
    temp = x[low];  
    x[low] = x[j];  
    x[j] = temp;  
    return j;  
}
```

```
void quick_sort(int a[], int low, int high)
```

```
{  
    int j;  
    if(low < high)  
    {
```

```

        j = partition(a, low, high);
        quick_sort(a, low, j-1);
        quick_sort(a, j+1, high);
    }
}

void main()
{
    int a[20], n, i;
    printf("Enter the size\n");
    scanf("%d", &n);
    printf("Enter the elements\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quick_sort(a, 0, n-1);

    printf("The sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
}

```

Sorting- Divide and Conquer

4. Divide-and-Conquer: Merge Sort

Idea

Merge sort follows the **divide-and-conquer technique**:

- Divide the array into **two halves**
- Recursively sort each half
- **Merge** the sorted halves into one sorted list

Characteristics

- Recursive algorithm
- Stable sorting technique
- Time complexity: **$O(n \log n)$** in all cases

C Program: Merge Sort

```
#include <stdio.h>
```



```
void mergeSort(int a[], int low, int mid, int high);  
void partition(int a[], int low, int high);
```

```
void main()  
{  
    int a[50], n, i;  
    printf("Enter total number of elements:");  
    scanf("%d", &n);  
  
    printf("Enter the elements:\n");  
    for(i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
  
    partition(a, 0, n - 1);  
  
    printf("After merge sort:\n");  
    for(i = 0; i < n; i++)  
        printf("%d ", a[i]);  
}
```

```
void partition(int a[], int low, int high)  
{  
    int mid;  
    if(low < high)  
    {  
        mid = (low + high) / 2;  
        partition(a, low, mid);  
        partition(a, mid + 1, high);  
        mergeSort(a, low, mid, high);  
    }  
}
```

```
void mergeSort(int a[], int low, int mid, int high)  
{  
    int i, l, m, k, temp[50];  
    l = low;  
    i = low;  
    m = mid + 1;  
  
    while(l <= mid && m <= high)  
    {  
        if(a[l] <= a[m])  
            temp[i++] = a[l++];  
        else
```

```

        temp[i++] = a[m++];
    }

    while(l <= mid)
        temp[i++] = a[l++];

    while(m <= high)
        temp[i++] = a[m++];

    for(k = low; k <= high; k++)
        a[k] = temp[k];
}

```

5. Decrease-and-Conquer: Insertion Sort

Idea

Insertion sort works by:

- Taking one element at a time
- **Inserting it into its correct position** in the already sorted part of the array

Characteristics

- Simple and efficient for small datasets
- Best case: $O(n)$ (already sorted)
- Worst case: $O(n^2)$ (reverse order)

C Program: Insertion Sort

```
#include <stdio.h>
```

```

void insertion_sort(int a[], int n)
{
    int i, j, temp;
    for(i = 1; i < n; i++)
    {
        temp = a[i];
        for(j = i - 1; j >= 0 && temp < a[j]; j--)
            a[j + 1] = a[j];
        a[j + 1] = temp;
    }
}

```

```

}

void main()
{
    int a[20], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    insertion_sort(a, n);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
}

```

6. Decrease-and-Conquer: Shell Sort

Idea

Shell sort is an **improved insertion sort**:

- Elements far apart are sorted first
- Uses a sequence of **decreasing increments (gaps)**
- Final pass uses increment = 1

Characteristics

- Faster than insertion sort
- Uses **simple insertion sort** on subfiles
- Time complexity depends on increment sequence

C Program: Shell Sort

```

#include <stdio.h>

void shell_sort(int a[], int n, int increments[], int numinc)
{
    int i, j, k, span, temp;

```

```

for(k = 0; k < numinc; k++)
{
    span = increments[k];
    for(i = span; i < n; i++)
    {
        temp = a[i];
        for(j = i - span; j >= 0 && temp < a[j]; j -= span)
            a[j + span] = a[j];
        a[j + span] = temp;
    }
}
}

```

```

void main()
{
    int a[20], n, i;
    int increments[] = {5, 3, 1};

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    shell_sort(a, n, increments, 3);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
}

```

Quick Exam Comparison (for conclusion)

Technique	Strategy	Time Complexity
Merge Sort	Divide & Conquer	$O(n \log n)$
Insertion Sort	Decrease & Conquer	$O(n^2)$

Shell Sort	Decrease & Conquer	Improved $O(n^2)$
------------	--------------------	-------------------

7.Time Complexity Comparison (Sorting Techniques)

Sorting Technique	Best Case	Average Case	Worst Case	Space Complexity	Remarks
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Simple exchange sort, inefficient for large data
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Number of comparisons fixed
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Efficient for small or nearly sorted lists
Shell Sort	$O(n \log n)^*$	$O(n \log n)^*$	$O(n^2)$	$O(1)$	Improved insertion sort, depends on increment sequence
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Divide-and-conquer, stable sort
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Very fast in practice, pivot-based

Searching

8.Searching – Sequential Search (C Program)

Sequential search is the **simplest searching technique** in which the elements of the table are **searched one by one from the beginning** until the required key is found or the list ends .

C Program: Sequential Search

```
#include<stdio.h>
```

```
int seq_search(int a[], int n, int key)
{
```

```

int i;
for(i = 0; i < n; i++)
{
    if(key == a[i])
        return i;
}
return -1;
}

void main()
{
    int a[20], n, i, key, pos;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter key to search: ");
    scanf("%d", &key);

    pos = seq_search(a, n, key);

    if(pos == -1)
        printf("Element not found");
    else
        printf("Element found at position %d", pos + 1);
}

```

Below is a **simple, exam-acceptable C program for Indexed Sequential Search**, written **strictly according to the concept explained in your notes** (index table + sequential search on block).

 Note (important for exam):

Your notes explain the **method and working**, not a fixed program.

This code is a **direct implementation of the same logic**, suitable if the question asks for a program.

9.C Program: Indexed Sequential Search

```

#include<stdio.h>

int main()
{
    int data[20], index[10];
    int n, i, key, block = 4;
    int indexSize, start, end, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements in sorted order:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &data[i]);

    /* Create index table */
    indexSize = n / block;
    for(i = 0; i < indexSize; i++)
        index[i] = data[i * block];

    printf("Enter key to search: ");
    scanf("%d", &key);

    /* Search in index table */
    for(i = 0; i < indexSize; i++)
    {
        if(key < index[i])
            break;
    }

    /* Define block range */
    if(i == 0)
        start = 0;
    else
        start = (i - 1) * block;

    end = start + block;
    if(end > n)
        end = n;

    /* Sequential search in block */
    for(i = start; i < end; i++)
    {
        if(data[i] == key)

```

```

    {
        printf("Element found at position %d\n", i + 1);
        found = 1;
        break;
    }
}

if(!found)
    printf("Element not found\n");

return 0;
}

```

How this matches your notes

- ✓ File is **sorted**
- ✓ **Index table** contains every *kth* element
- ✓ First search on **index**
- ✓ Then **sequential search** on a small block
- ✓ Same logic described under *Indexed Sequential Search*

10.Binary Search – C Program (Iterative)

```

#include<stdio.h>

int binary_search(int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(a[mid] == key)
            return mid;
        else if(key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
}

```



```

    return -1;
}

void main()
{
    int a[20], n, i, key, pos;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements in sorted order:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter key to search: ");
    scanf("%d", &key);

    pos = binary_search(a, n, key);

    if(pos == -1)
        printf("Element not found");
    else
        printf("Element found at position %d", pos + 1);
}

```

11. Interpolation Search

```

#include<stdio.h>

int interpolation_search(int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;

    while(low <= high && key >= a[low] && key <= a[high])
    {
        mid = low + (high - low) * ((key - a[low]) / (a[high] - a[low]));

        if(a[mid] == key)
            return mid;
        if(key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
}

```

```
    return -1;
}

void main()
{
    int a[20], n, i, key, pos;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements in sorted order:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter key to search: ");
    scanf("%d", &key);

    pos = interpolation_search(a, n, key);

    if(pos == -1)
        printf("Element not found");
    else
        printf("Element found at position %d", pos + 1);
}
```